FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433 SOFTWARE ARCHITECTURE

PROJECT TITLE:

CLOUD NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

GROUP 4

Name	Matric No.
EASHENRAJ A/L NAGARAJAN	S76773
MOHAMMAD ADAM BASHEER B. MOHD AZRUL	S75710
AMMAR HAZIQ B. SAIFUL BAHRI	S75772

PREPARED FOR:

DR MOHAMAD NOR HASSAN

Table of Contents

1. Introduction

The Learning Management System (LMS) is a platform designed to manage educational activities which consist of course creation, enrollment, content delivery, assignment submission, and grading. This system follows the Layered Architecture Pattern, which ensures the separation of concerns between the presentation layer, business logic layer, data access layer, and database layer.

This architectural pattern improves the system's maintainability, scalability, and security by isolating each layer's responsibilities. The presentation layer is built using React and deployed on Netlify, the business logic layer is developed using Spring Boot and deployed on Render, the data access layer is implemented through Spring Data JPA and Hibernate running within the same Spring Boot container, and the database layer is hosted on Aiven MySQL as a cloud-managed relational database.

The system uses RESTful API endpoints to ensure structured communication between the frontend and backend. The React frontend communicates with the Spring Boot backend exclusively through these endpoints using Axios as the HTTP client, sending and receiving data in JSON format over HTTPS. These endpoints can be tested and validated using API testing tools such as Postman.

To provide an overview of this application, this technical report is structured as follows. Section 2 and Section 3 outline the core problem description and the system's functional and non-functional requirements. Section 4 and Section 5 detail the architectural design choices, component responsibilities, and structural diagrams (including context, sequence, and deployment diagrams). Finally, Sections 6 through 10 discuss the underlying technology stack, prototype implementation details, architectural evaluation metrics, and the team's reflections on the development lifecycle.

2. Problem Description

Traditional Learning Management Systems (LMS) often suffer from a monolithic and tightly coupled design where the user interface, business logic, and database are intertwined with each other, causing several problems.

In traditional LMS designs, the user interface, business logic, and database queries are usually associated with each other. This makes system maintenance more complex. As a result, updating one feature may unintentionally break other features, leading to long downtimes and high risks during system updates. This is the core problem our system addresses by separating each responsibility into its own distinct layer: React handles only the UI, Spring Boot handles only the business logic, and Aiven MySQL handles only data storage.

The absence of a dedicated Data Access Layer (DAL) in traditional LMS designs causes tight coupling between the user interface, business logic, and database. Accessing features such as course materials and submitting assignments may directly interact with the database, which risks data corruption and makes the system less reliable and harder to scale. In our system, this is prevented by implementing Spring Data JPA repositories as the dedicated DAL, ensuring that all database interactions are routed exclusively through structured DAO interfaces rather than directly from the business logic or UI.

Lastly, traditional LMS designs create scaling problems because the system is not modularized. Peak academic periods, such as course registration, cause traffic problems when students attempt to access the database simultaneously, potentially crashing the platform. Our system addresses this by deploying each layer independently. The frontend on Netlify, the backend on Render, and the database on Aiven, allowing each layer to be scaled or updated without affecting the others.

3. System Requirements

3.1 Functional Requirements

To effectively demonstrate a layered, cloud-native architecture, the system's functional requirements are categorized into four modules. This ensures the separation of concerns across the presentation, business logic, and data access layers.

3.1.1. User Identity and Access Management Module

In the User Registration the system allows new users to create an account by providing basic details such as name, email, matric number, academic programme, password, and role. Registration is handled by the UserController and processed through UserService, which validates the uniqueness of the matric number and email before saving to the users table in Aiven MySQL.

a. **Authentication**

   The system securely authenticates users upon login by verifying the submitted password against the BCrypt hashed password stored in the database using Spring Security Crypto. Upon successful authentication, the user object including role is returned and stored in the browser's localStorage to manage session access across the application.

b. **Role-Based Access Control**

   The system enforces role-specific permissions based on the user's role enum value of either STUDENT or INSTRUCTOR. This prevents students from accessing instructor-only features such as course creation and assignment grading at both the frontend routing level in React and the backend service level in Spring Boot.

c. **Profile Management**

   Users are able to view their personal profile information including name, matric number, and programme displayed on their respective dashboards.

3.1.2. Course Management Module

a. Course Creation

The system allows instructors to create a new course by defining a course name, course code, description, and enrollment capacity. This is handled by CourseController via a POST /api/courses endpoint and processed through CourseService before being persisted by CourseRepository to the courses table.

b. Course Catalog

The system displays a catalog of all available courses to students through a GET /api/courses endpoint, rendered as course cards on the student dashboard showing the course code, name, instructor, and available capacity.

c. Enrollment Processing

The system allows students to enroll in available courses through a POST /api/courses/{courseId}/enroll endpoint. The CourseService enforces business rules including role validation, capacity checks, and duplicate enrollment prevention. Upon successful enrollment, the student's dashboard is automatically updated to reflect their active enrollments under a dedicated My Courses tab.

3.1.3. Content Delivery and Management Module

a. Material Upload

The system allows instructors to upload educational materials such as links to external resources, lecture notes, or video links to specific courses. Each content item is stored with a title, content type, and URL reference linked to the relevant course in the contents table.

b. Material Access

The system allows enrolled students to view course materials through a GET /api/contents/course/{courseId} endpoint, retrieving all content associated with their enrolled courses from the database.

3.1.4. Assignment and Assessment Module

a. Assignment Creation

Instructors are able to create assignments by specifying a title, instructions, maximum score, and a strict submission deadline. This is handled through a POST /api/assignments endpoint and persisted to the assignments table linked to the relevant course.

b. Submission Tracking

The system automatically tracks and assigns a submission status of SUBMITTED, LATE, PENDING, or OVERDUE based on whether the submission timestamp falls before or after the assignment due date. This logic is enforced in SubmissionService.

c. Grading and Feedback

Instructors are able to view student submissions, assign a numerical grade, and provide text-based feedback through a PUT /api/submissions/{submissionId}/grade endpoint, which updates the submission record and makes the grade and feedback visible to the respective student.

3.1.5. System and Architectural Requirements

a. API Communication

The system's React frontend communicates with the Spring Boot backend exclusively through a well defined RESTful API endpoints using Axios as the HTTP client, sending and receiving data in JSON format over HTTPS.

b. Data Persistence

The system accurately stores and retrieves all user, course, content, assignment, and submission data using Aiven MySQL as the cloud-hosted relational database, managed through Spring Data JPA and Hibernate ORM.

3.2 Non-Functional Requirements

3.2.1. Performance

Performance ensures the system remains responsive under pressure, prioritizing consistency over just speed.

a. The system must support a minimum of 100 concurrent users without significant degradation in response time. This is supported by deploying the Spring Boot backend on Render, which handles concurrent HTTP requests through Apache Tomcat's built-in thread pool.

b. RESTful API endpoints should return responses within 2 seconds under normal operating conditions. This is achieved by using Spring Data JPA with Hibernate ORM to execute optimized database queries against Aiven MySQL, minimizing unnecessary data retrieval.

3.2.2. Scalability

Scalability is how the system can handle growth without changing its overall design.

a. The system supports horizontal scaling by deploying each layer on independent cloud platforms. The React frontend is hosted on Netlify, the Spring Boot backend is hosted on Render, and the MySQL database is hosted on Aiven, meaning each layer can be scaled independently without affecting the others.

b. The Layered Architecture enables each layer such as the Business Logic Layer or Data Access Layer to be scaled independently based on demand, since no layer is directly dependent on the physical infrastructure of another.

3.2.3 Security

This non-functional requirement prioritizes data integrity and privacy, which ensures confidentiality.

a. Role-Based Access Control is enforced at both the frontend and backend levels. React routes users to either the Student dashboard or the Instructor console based on the role value returned from the backend. Spring Boot service methods validate the user role before executing sensitive operations such as course creation or grading.

b. User passwords are stored using the BCrypt hashing algorithm provided by Spring Security Crypto, ensuring plain-text passwords are never stored in the Aiven MySQL database or transmitted over the network.

c. All communication between the React frontend and the Spring Boot backend is secured over HTTPS, and cross-origin requests are controlled through the CORS configuration defined in WebConfig.java.

3.2.4. Availability

Availability measures the time the system is fully operational and accessible, also known as uptime. This ensures business continuity, reliability, and high performance.

a. The system targets a minimum uptime of 99% following deployment. Netlify provides continuous static hosting with no downtime, Aiven MySQL operates as a fully managed 24/7 cloud database, and Render keeps the backend service live with automatic restarts on failure.

b. System updates and bug fixes are deployable at the individual layer level since each layer is hosted independently. Updating the frontend on Netlify or the backend on Render does not require restarting the database, minimizing overall system downtime.

3.2.5 Maintainability

Maintainability refers to the ease with which the system can be understood, updated, and extended over time.

a. The Layered Architecture separates the system into four independent layers that includes Presentation (React on Netlify), Business Logic (Spring Boot on Render), Data Access (JPA repositories and Hibernate within Spring Boot), and Database (Aiven MySQL) allowing each to be modified or tested without impacting the others.

b. Loose coupling between components is achieved through the REST API contract between the frontend and backend, and through the DAO pattern in the repository layer. Changes to the UI do not affect the service layer, and changes to the database schema only require updates to the model and repository packages without touching the business logic or frontend code.

4. Architecture Design

4.1 Selected Architecture Style

The architecture style used in this project is the Layered Architecture Pattern. This design consists of different logical layers for an application, where each layer has a defined responsibility and communicates only with its adjacent layers. This approach improves maintainability because components can be modified more easily, and scalability by allowing the system to scale different parts separately. This architecture minimizes dependencies between layers, enabling greater flexibility and easier modification of individual components. For this LMS, we are using a closed layered approach, which means a layer cannot bypass another layer to access the lower layer.

a. **The Presentation Layer**

   This is responsible for user interface design and user experience. This layer handles visual displays and user input, then communicates with the Business Logic Layer to display processed results. In this system, the presentation layer is built using React with Vite and deployed on Netlify.

b. **The Business Logic Layer**

   This is the layer where information received from the previous layer is processed. This layer handles complex operations and core business rules such as user authentication, enrollment validation, and submission deadline tracking. Having a separate layer allows modification to business rules without changing the user interface or database. In this system, the business logic layer is implemented using the Spring Boot framework and deployed on Render.

c. **The Data Access Layer**

This is where data interactions happen. This layer ensures communication between the system and data storage by handling CRUD operations and acting as a gatekeeper to ensure only authorized requests reach the data. In this system, the data access layer is implemented using Spring Data JPA repositories and Hibernate ORM, running within the same Spring Boot container as the business logic layer.

d. **The Database Layer**

This layer consists of the actual database server storing all persistent system data. This layer is designed to be independent, utilizing a cloud database to improve reliability. In this system, the database layer is hosted on Aiven MySQL as a fully managed cloud relational database.

This architecture pattern allows the project to achieve Separation of Concerns, where each layer can be modified without making changes to other layers. It also allows each layer to be tested independently, which helps reduce the risk of bugs in the system.

4.2 Service and Component Identification

a. **Presentation Layer**

This layer is responsible for handling every user interaction by displaying the system user interface. It is developed using React with Vite, which provides dynamic and responsive user interfaces such as the login page, student dashboard, and instructor console. Axios is used as the HTTP client to send RESTful API requests from the frontend to the backend. Deployment is handled by Netlify, which provides fast content delivery, high availability, and continuous deployment from GitHub.

b. Business Logic Layer

This layer handles the processing of application logic. It is implemented using the Spring Boot framework, which exposes RESTful API endpoints for communication with the frontend. This layer handles the core system functionalities through four service components which are UserService for authentication and registration, CourseService for course creation and enrollment processing, AssignmentService for assignment creation and submission tracking, and ContentService for course material management. It is deployed on Render, which provides cloud hosting for backend services with automatic restarts and reliable uptime.

c. Data Access Layer

This layer manages the communication between the backend and the database. It implements the DAO pattern using Spring Data JPA repository interfaces to separate the database logic from the business logic. The four repository components which are UserRepository, CourseRepository, AssignmentRepository, and ContentRepository handle all CRUD operations and ensure the consistency and security of the system data. Hibernate ORM acts as the bridge between the Java model classes and the MySQL database tables.

d. Database Layer

This layer is responsible for data storage for the LMS system. Aiven MySQL is used as the cloud database service, storing structured data across five tables which is users, courses, assignments, contents, and submissions. It provides automated backups, high availability, and secure SSL-based database access.

Component	Responsibility	Exclusions
Web Client (React SPA/Vite)	• Renders the user interface • Get user input • Sends HTTP requests to the API Gateway • Displays responses	• Does not process business logic • Does not access the database directly
API Gateway	Routes incoming HTTPS REST requests to the correct backend service	• Does not perform business logic • Does not store data
Auth Service	Handles user login, registration, JWT token generation, and validation	Does not manage course, assignment, or content data
Profile Service	Manages the viewing and updating of user personal information.	Does not handle authentication
Course Service	Handles course creation, modification, deletion, search, and enrollment processing	Does not handle assignment submission or file content uploads
Assignment Service	Handles assignment creation, file submission, deadline checking, submission status tracking (Submitted, Late, Pending, Overdue), grading, and feedback.	Does not manage course content or user profiles

Content Service	Manages the upload, and delivery of course materials	Does not handle authentication or assignment deadlines
User DAO	Executes CRUD operations on the Users table for user accounts, profiles, and roles	● Does not contain business logic ● Does not access any other table
Course DAO	Executes CRUD operations on the Courses table for course details and schedules	● Does not contain business logic ● Does not access any other table
Assignment DAO	Executes CRUD operations on the Assignments table for assignments, submissions, grades, and feedback	● Does not contain business logic ● Does not access any other table
Content DAO	Executes CRUD operations on the Contents table for content metadata and file references	● Does not contain business logic ● Does not access any other table
Aiven MySQL	Persists all structured data across four tables: Users, Courses, Assignments, Contents	● Does not process application logic ● Does not communicate directly with the frontend

Table 4.2 : Component Responsibilities and Exclusions

5. Architecture Diagrams

This section details the blueprint of the Learning Management System, providing a comprehensive overview of how different components, data flows, and infrastructure elements work together. By moving from high-level system boundaries to specific runtime interactions and physical deployment, these diagrams ensure a clear understanding of the application's scalability and modular design.

5.1 System Context Diagram

This system context diagram illustrates the boundaries of the Learning Management System, showing how it interacts with two external actors which is Student and Instructor while integrating with cloud infrastructure for hosting and data management. It provides a high-level view of the entire ecosystem, detailing the data exchange between the core system and external entities to ensure seamless academic operations.

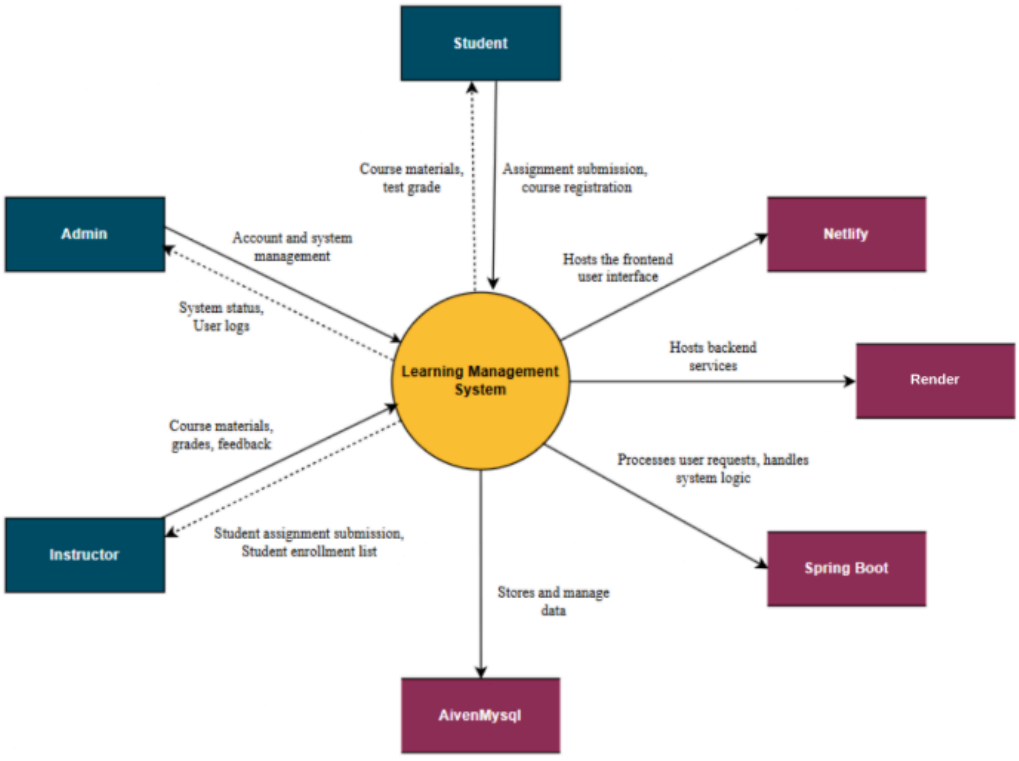

Figure 5.1: System Context Diagram

5.2 Component and Service Architecture Diagram

The component diagram for the Learning Management System illustrates a modular layered architecture, detailing the internal service components that power the platform's business logic. The four core service components which are User and Auth Service, Course Management Service, Assignment and Assessment Service, and Content Delivery Service will communicate with their corresponding DAO components in the Data Access Layer through JPA repositories and Hibernate ORM. It provides a technical breakdown of how RESTful API endpoints exposed by the Spring Boot controllers bridge the React frontend with the backend services to ensure organized data flow and system reliability.

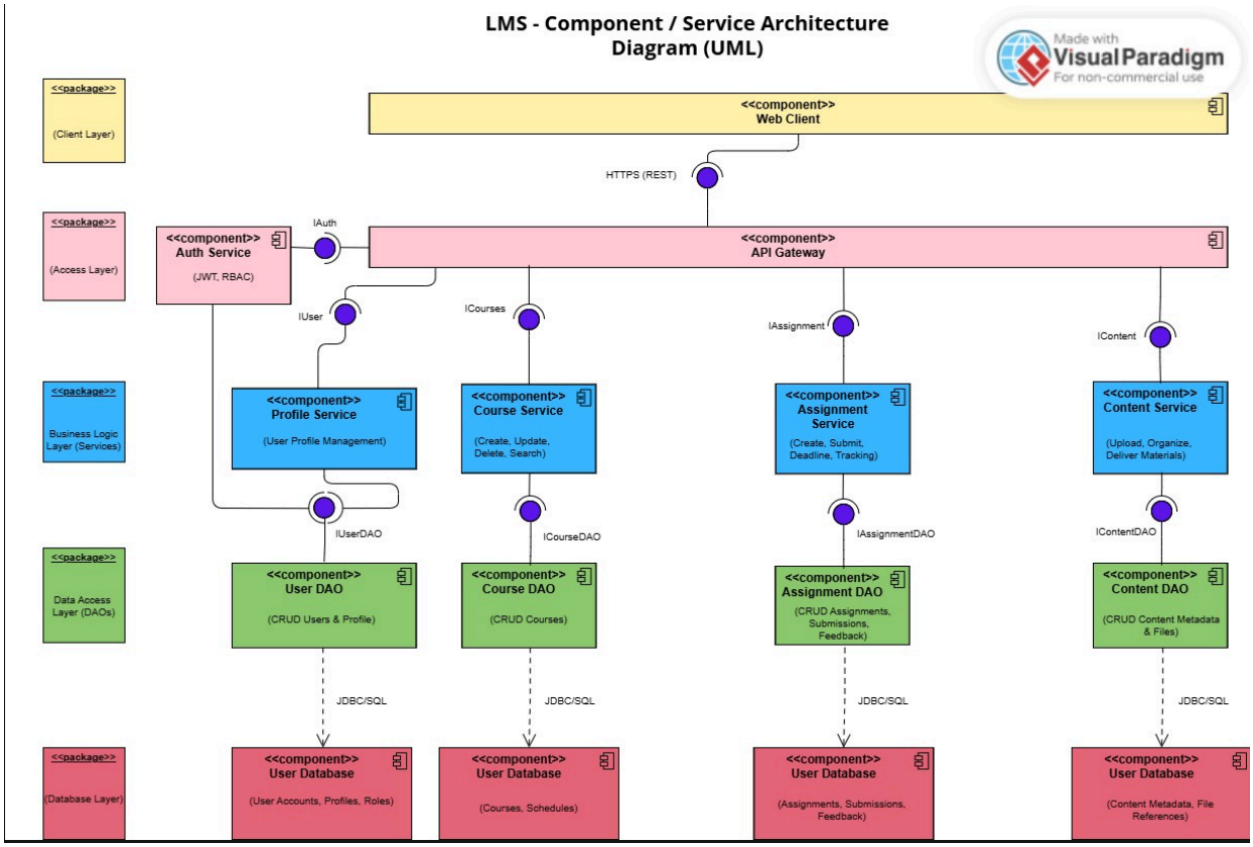

Figure 5.2: Component/Service Architecture Diagram

5.3 Interaction Diagrams

5.3.1 User Authentication Sequence Interaction

Figure 5.3.1 visualizes the step-by-step interaction between the user and the backend systems, outlining how login credentials are sent from the React frontend via Axios to the UserController, which passes them to UserService for BCrypt password verification against the stored hash in Aiven MySQL. It highlights the conditional logic used to return the user object with role information upon successful verification, or return an error message if authentication fails, ensuring a secure and responsive login flow.

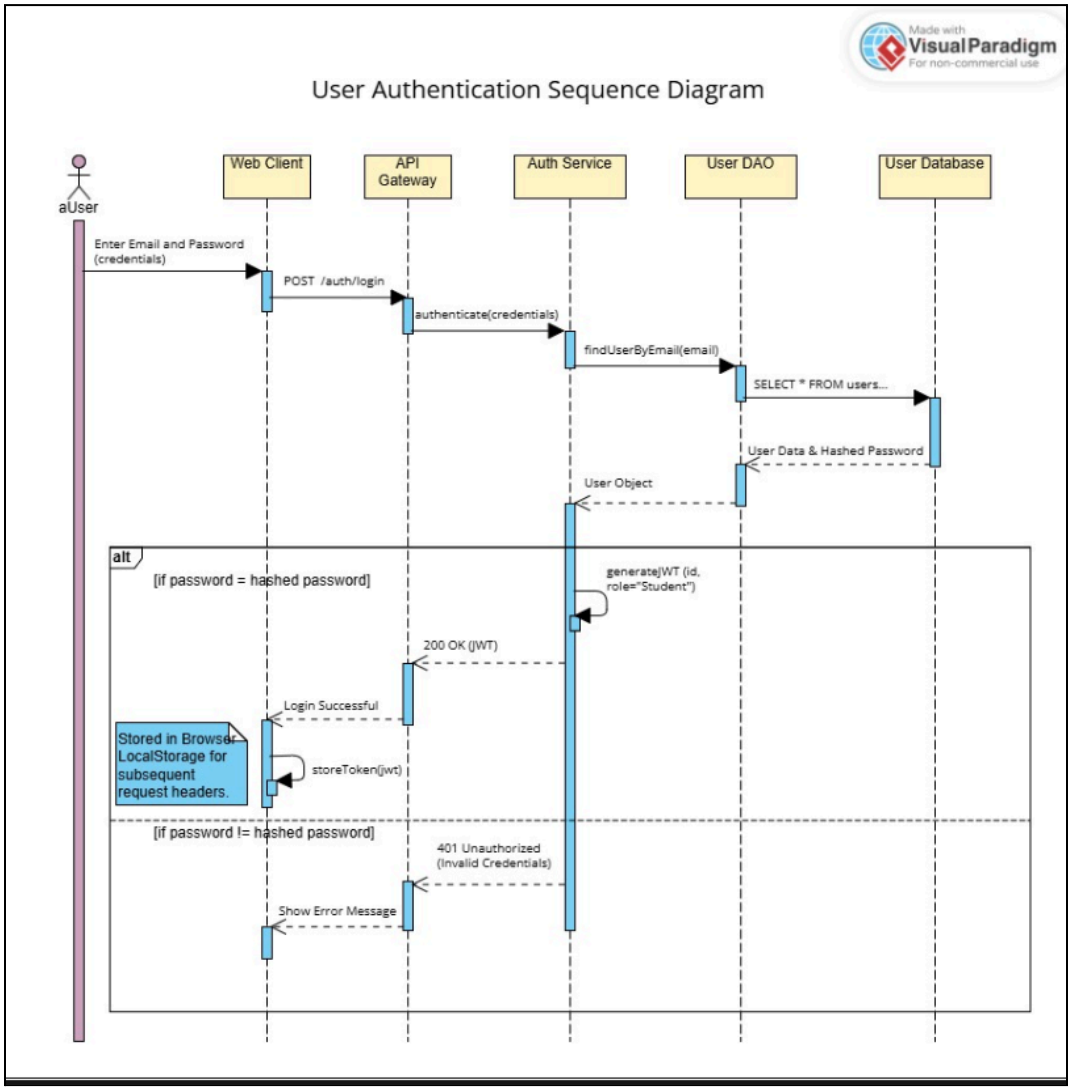

Figure 5.3.1 User Authentication Sequence Diagram

5.3.2 Update Course Interaction

Figure 5.3.2 depicts the workflow for instructors to update an existing course, highlighting how the React frontend sends a PUT /api/courses/{courseId} request with the updated details to CourseController. It follows the data path through CourseService to CourseRepository, which updates the existing course record in the courses table, confirming that the changes are only persisted after verifying the instructor's role and ownership of the course.

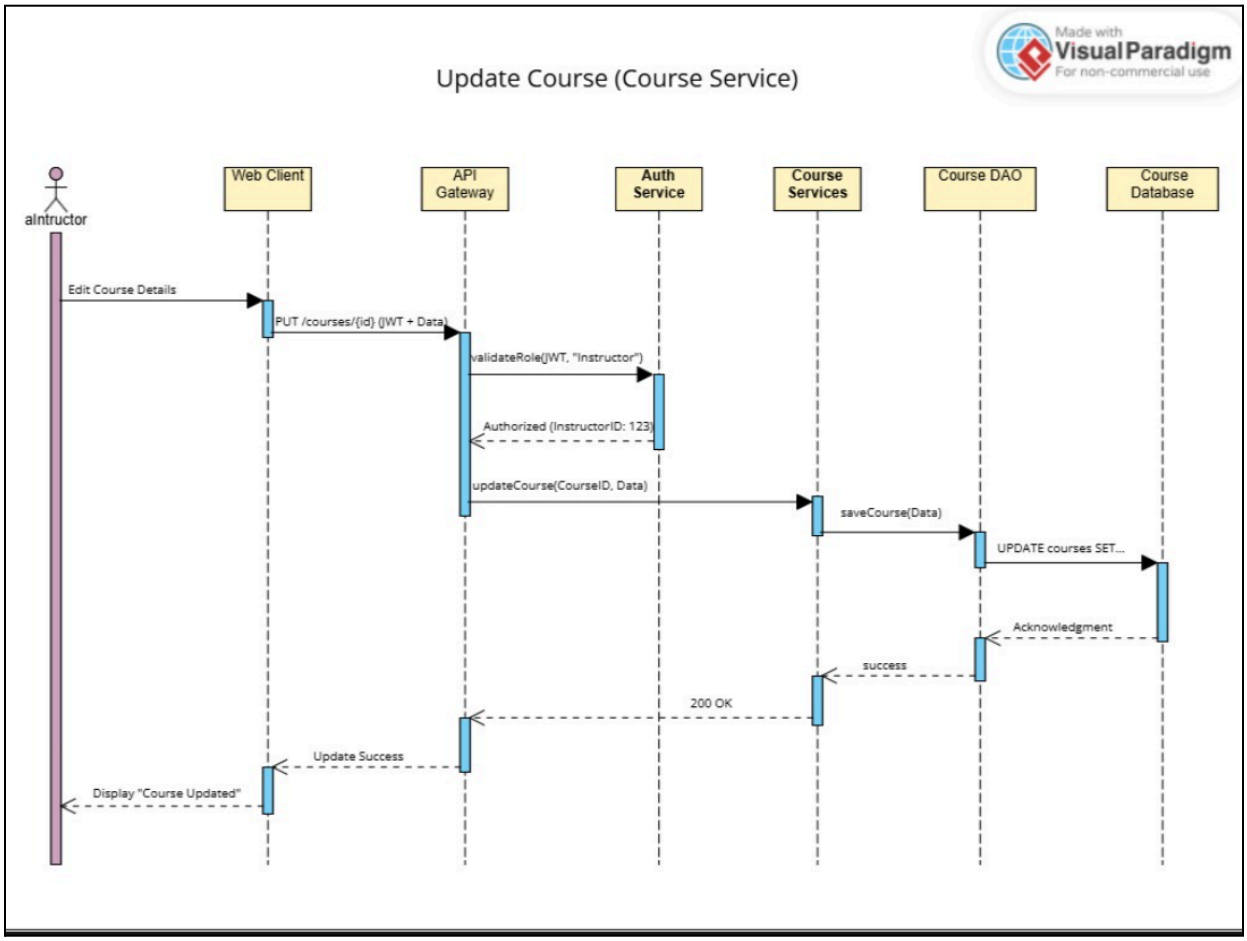

Figure 5.3.2 Update Course Sequence Diagram

5.3.3 Submit Assignment Interaction

Figure 5.3.3 Illustrates the workflow for students to submit an assignment and the automated validation logic in SubmissionService that checks the current timestamp against the assignment due date. It details how the submission status is automatically set to SUBMITTED or LATE based on this check before the SubmissionRepository persists the submission record to the submissions table in Aiven MySQL.

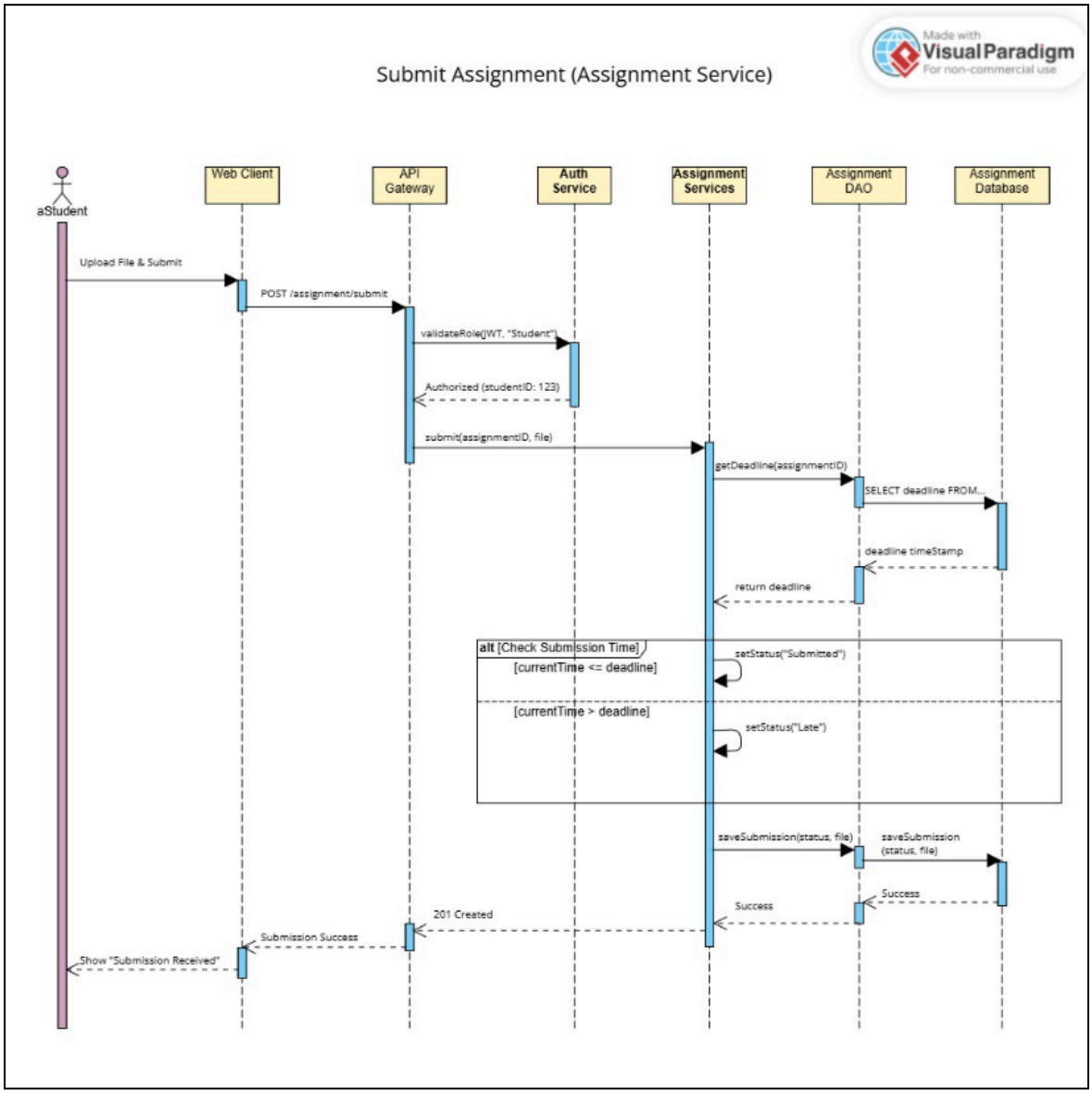

Figure 5.3.3 Submit Assignment Sequence Diagram

5.4 Deployment Diagram

The deployment diagram provides a comprehensive map of the system's physical infrastructure, illustrating how the React frontend deployed on Netlify communicates with the Spring Boot backend hosted on Render via RESTful API calls over HTTPS using Axios. It highlights the separation of concerns across the four layers which is the presentation layer on Netlify, the business logic and data access layers within the Spring Boot container on Render, and the database layer on Aiven MySQL that culminates in a secure JDBC connection for persistent relational data storage.

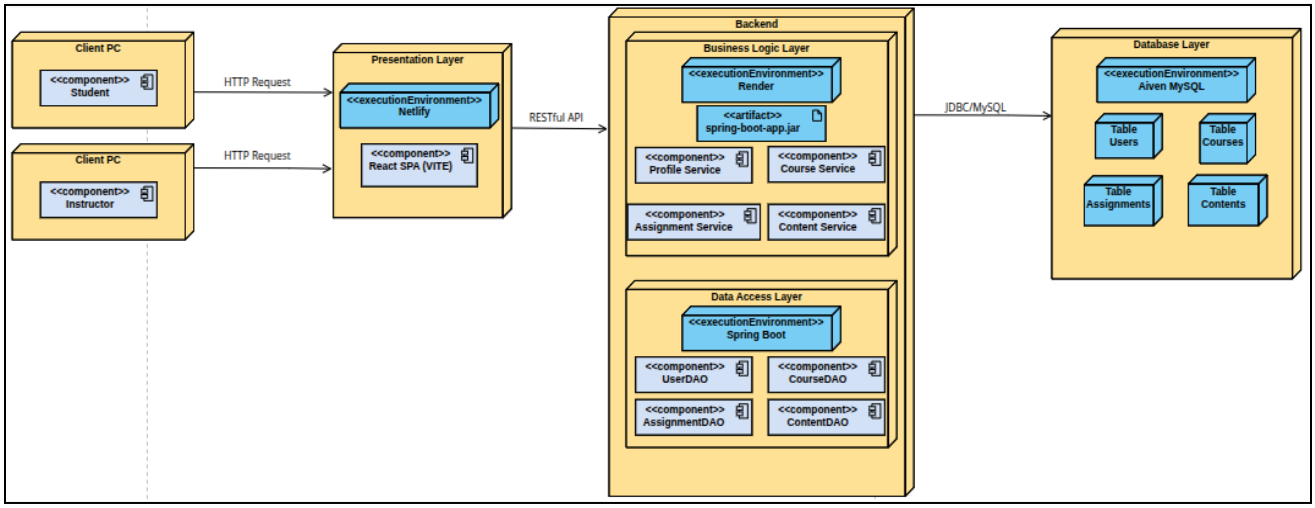

Figure 5.4 Deployment Diagram

6. Technology Stack

At the Frontend, React with Vite is used to build the Single Page Application, providing a dynamic and responsive user interface across the login page, student dashboard, and instructor console. Axios is used as the HTTP client library to send RESTful API requests to the backend. The frontend is hosted and deployed on Netlify with continuous deployment from GitHub. While in the Backend, The Spring Boot is used as the main Java framework, running on an embedded Apache Tomcat server on port 8080. It exposes RESTful API endpoints through five controller classes, which are UserController, CourseController, AssignmentController, ContentController, and SubmissionController, and processes all business logic through their corresponding service classes. Spring Security Crypto provides BCrypt password hashing for secure user authentication. The backend is hosted and deployed on Render.

For the Data Access Layer, the DAO pattern is implemented using Spring Data JPA repository interfaces, which are UserRepository, CourseRepository, AssignmentRepository, ContentRepository, and SubmissionRepository. Hibernate ORM acts as the bridge between the Java entity model classes and the MySQL database tables, automatically handling CRUD operations and table creation through the spring.jpa.hibernate.ddl-auto=update configuration. All database communication is handled exclusively through these repository interfaces, ensuring the business logic layer never directly accesses the database.

For the database layer, Aiven MySQL is used as the cloud-hosted relational database. It stores all persistent system data across five tables (users, courses, assignments, contents, and submissions). The connection is secured using SSL with the JDBC connector configured in application.properties. Aiven provides automated backups, high availability, and 24/7 managed uptime. Docker is used for containerization during local development to run a containerized MySQL database instance on port 3306, providing an isolated and consistent development environment before connecting to the Aiven cloud database for production deployment. Postman is used as the API testing platform to validate all RESTful API endpoints during development, verifying correct request and response behaviour across registration, login, course management, enrollment, content upload, and assignment submission flows.

7. Prototype Implementation

This section contains the system codebase implementation details, sample application routes, endpoint execution records, and frontend views corresponding to the prototype features. The prototype demonstrates a fully functional cloud-native LMS deployed across three platforms which is Netlify for the frontend, Render for the backend, and Aiven MySQL for the database with all four architectural layers operational end to end.

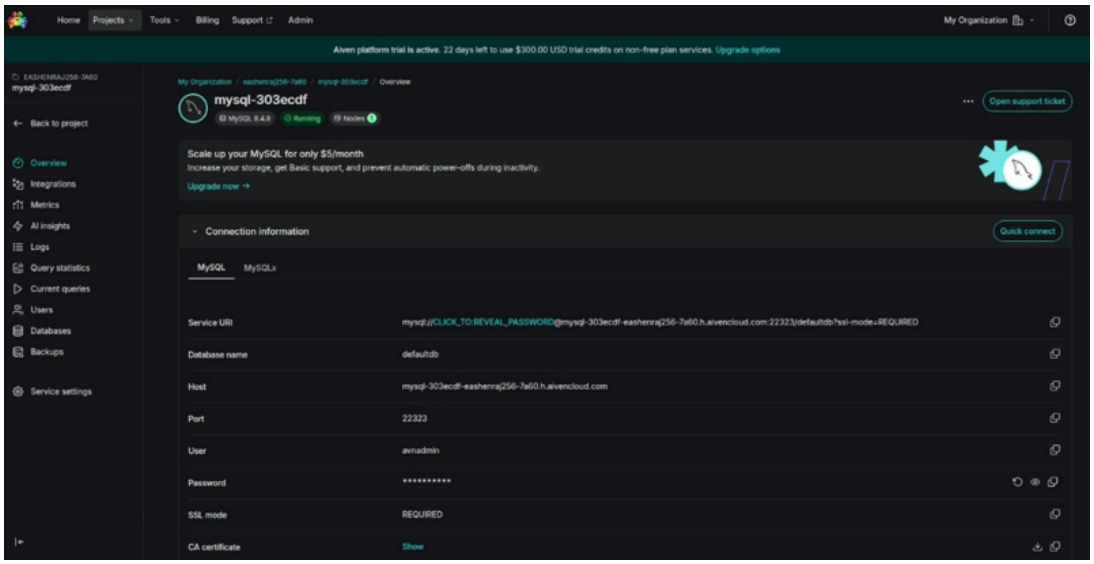

Figure 7.1: Aiven MySQL for backend

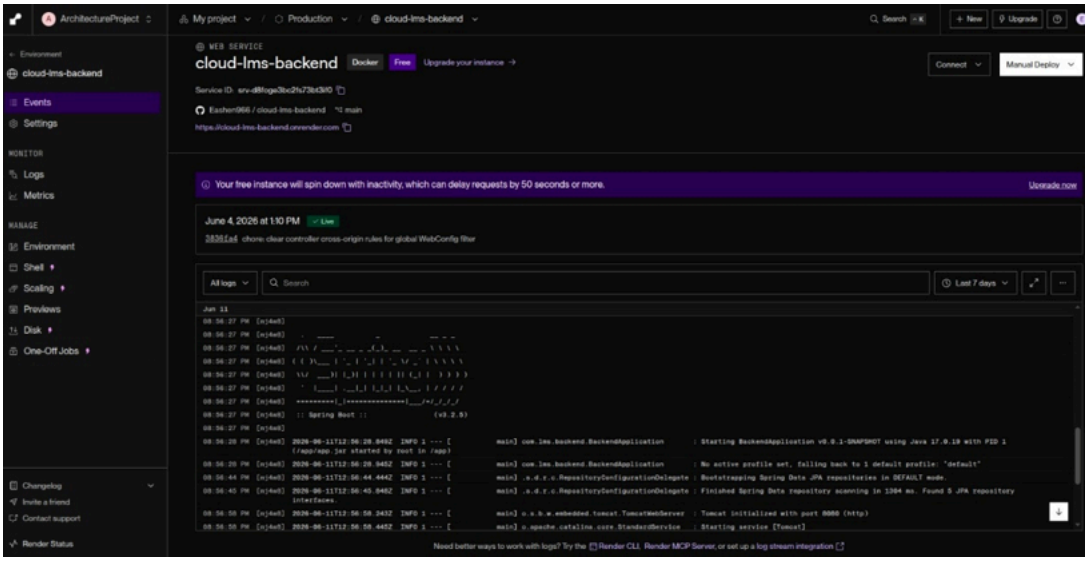

Figure 7.2: Cloud LMS backend

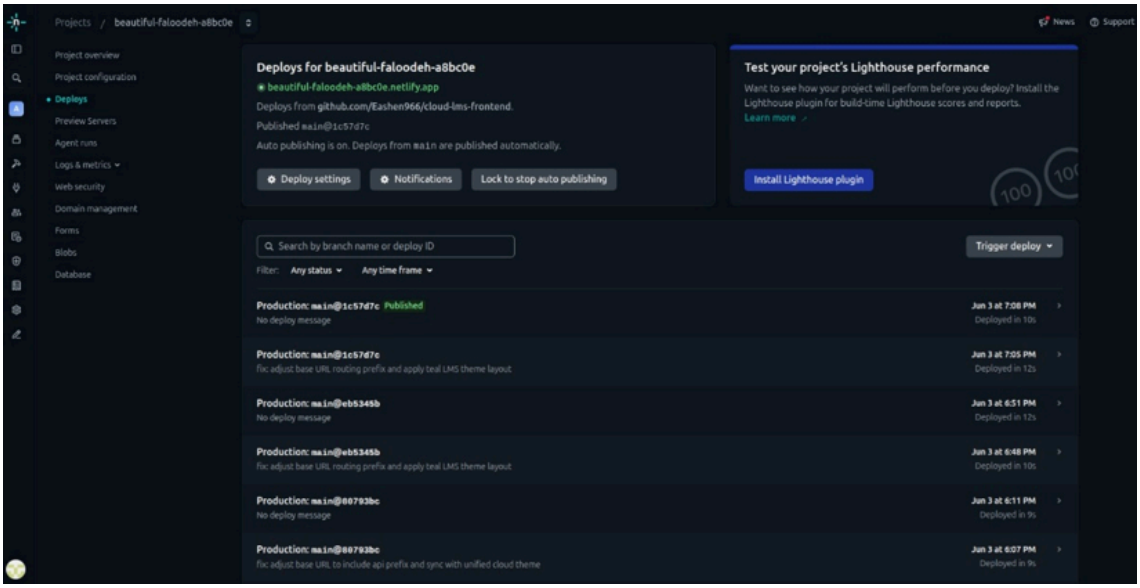

Figure 7.3: Netlify frontend deployment

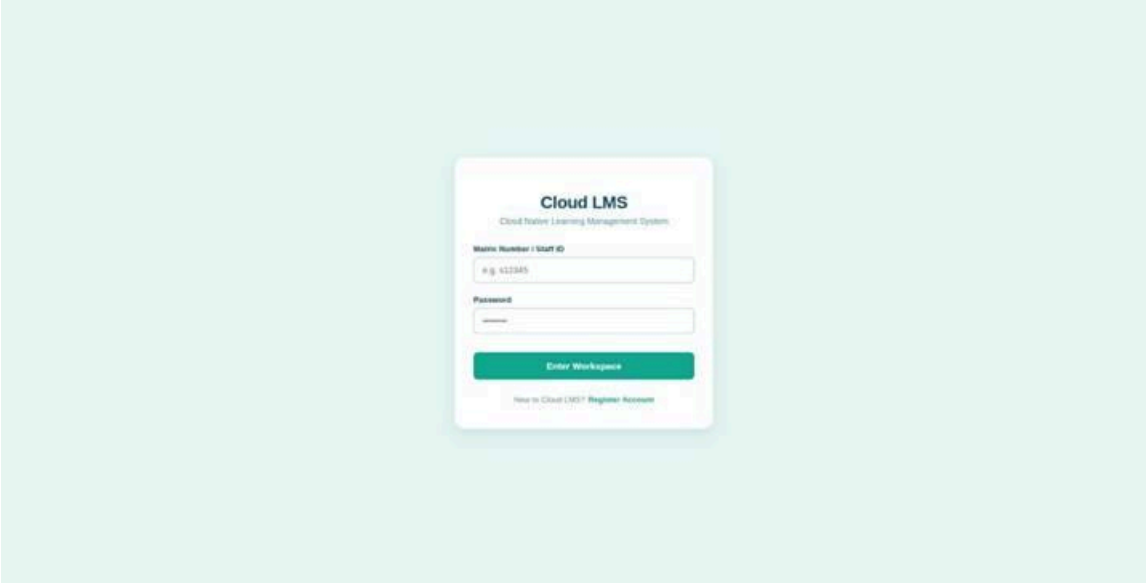

Figure 7.4: Login User Interface

Figure 7.4 shows the login interface of the Cloud LMS. Users can access the platform through login by entering their matric number (for students) or staff ID (for instructors) with their registered passwords.

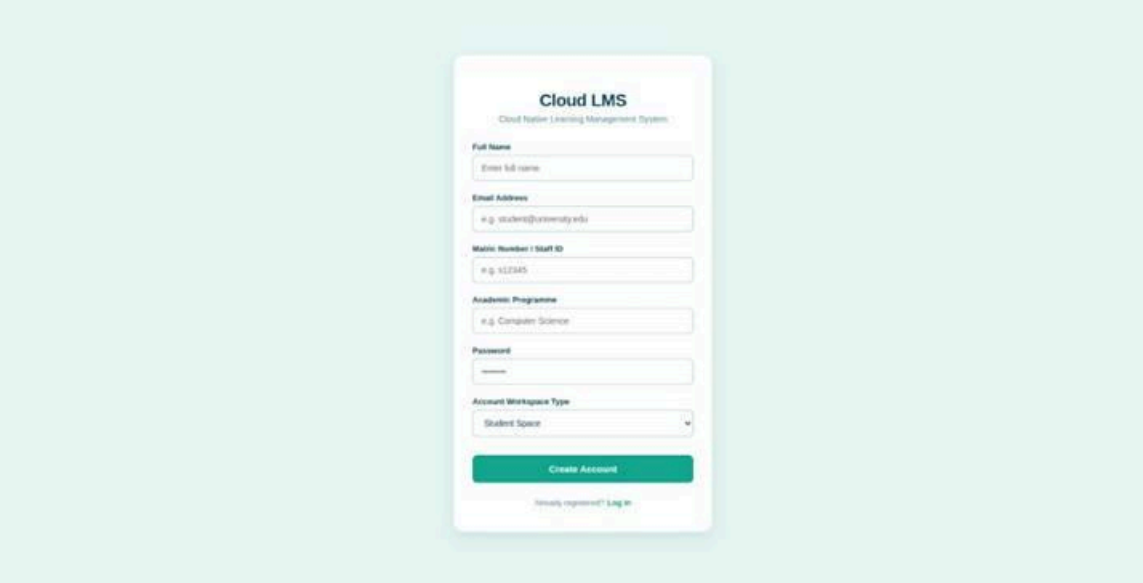

Figure 7.5: Registration User Interface

Figure 7.5 displays the user interface for the registration form. The users can register their account into the system. They can register by entering their full name, email, academic programme and account workspace type.

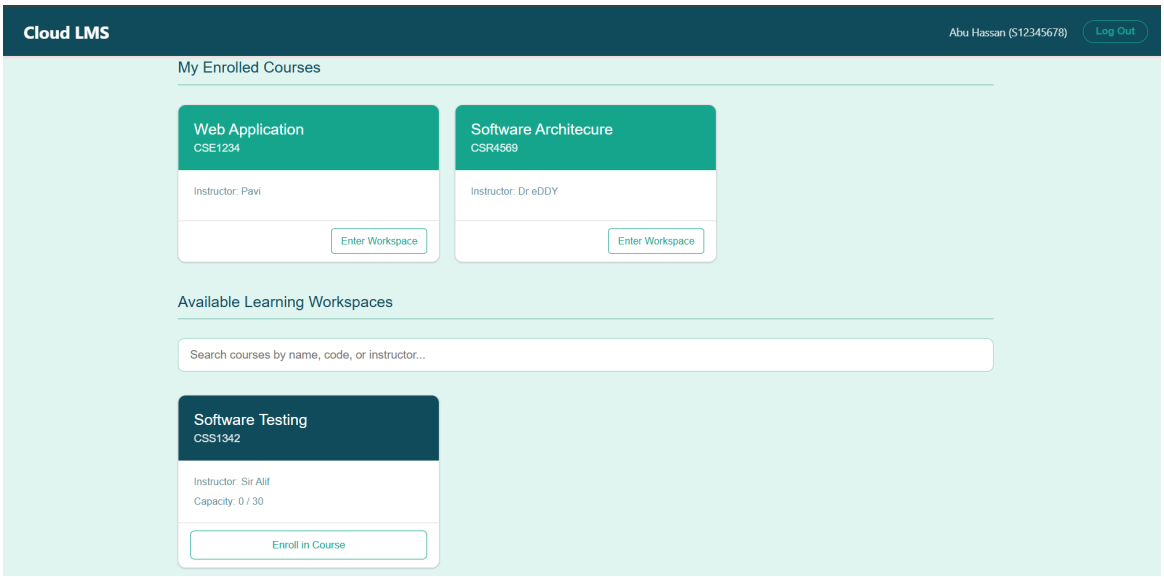

Figure 7.6: Student Dashboard Interface

Figure 7.6 is the dashboard of the system that displays all the available courses for students to enrol. Each course will provide information such as course code, course name, and capacity.

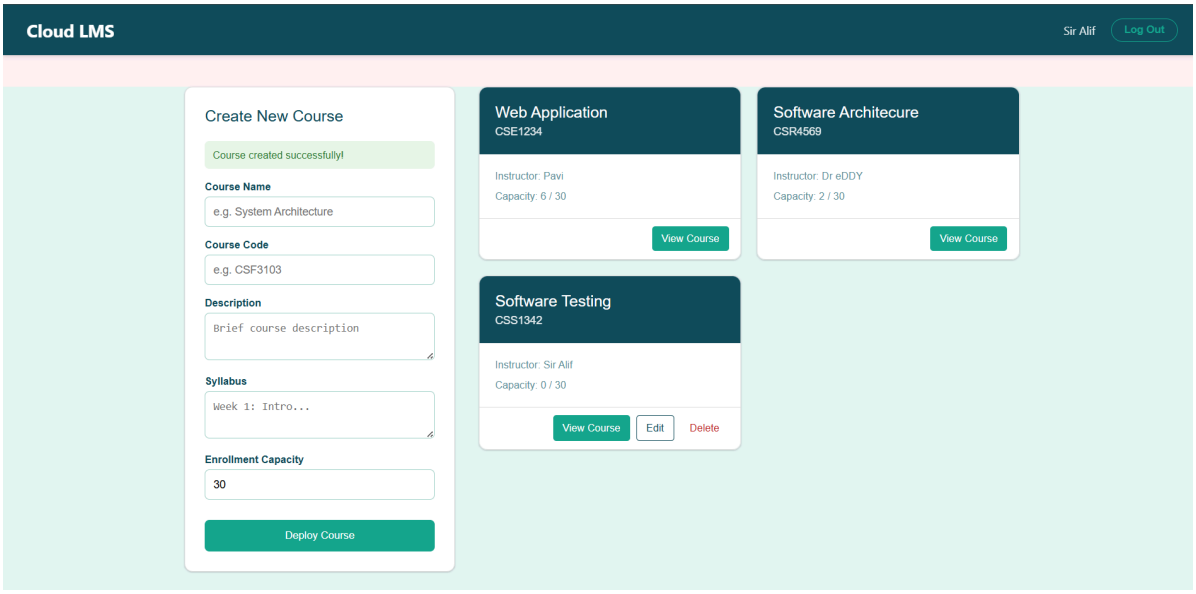

Figure 7.7: Instructor Dashboard Interface

Figure 7.7 shows the interface where the instructor uses to create a new course. They can input the essential course details such as course name, course code, description and the enrolment capacity of students. Created courses can be managed by the creator to edit details, update and delete the created course. Each created course will be displayed to the students through the student dashboard.

8. Architecture Evaluation

This section documents the structural validation of the implemented system against the defined architectural pattern and non-functional requirements. The evaluation verifies that the four-layer separation of concerns is correctly enforced across the presentation, business logic, data access, and database layers, and that the system behaves as expected under the defined functional and non-functional requirements.

Performance: Evaluated the cloud lms system performance using AutoCannon to ensure a good response time. The result shows a slight difference in response time which exceeded 2 seconds.

```
PS C:\lms> npx autocannon -c 100 -d 15 https://cloud-lms-backend.onrender.com/api/courses
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	3417 ms	3553 ms	4827 ms	4827 ms	3915 ms	550.8 ms	4827 ms

Figure 8.1: Evaluation of Cloud LMS performance

Scalability: Implemented the use of multiple tier architecture by using different layers for each entity. This ensures that each service can be scaled independently without breaking the system functionality.

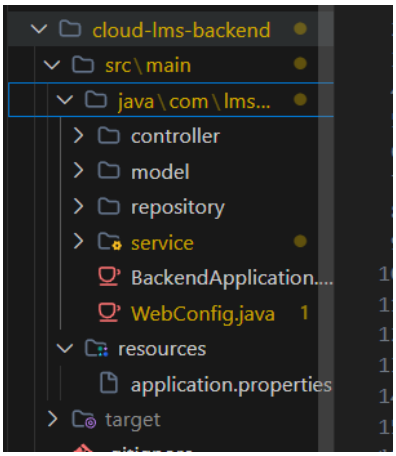

Figure 8.2: Separation of layers into controller, model, repository and service

Security: Prioritized data integrity and user privacy by implementing BCrypt password hashing. This mitigated the risk of brute-force attacks and ensured that all sensitive credentials were fundamentally secure at rest.

	user_id int	* email varchar(100)	* matric_no varchar(20)	* name varchar(100)	* password_hash varchar(255)	* programme varchar(100)
	1	pavi@umt.com	a123	Pavi	$2a$10$1zswrV.DobWty5UJ	cs
	2	jev@umt.com	s123	Jeevan	$2a$10$Jp9lsE8zR4aZpwF0(	cs
	3	eashen@umt.com	S76773	Eashenraj	$2a$10$D1SC28n4Le2lXpua	Computer Science
	4	pavitranselvaraju101@gmai	S75310	PAVITRAN A/L SELVARAJU	$2a$10$j5BA9FMebgNMGd	Computer Science
	5	PavitronManap@ocean.umt	S67670	Pavitron Abdul Manap	$2a$10$7wkeSPC0VZcE6TX	Fisheries
	6	nmqasrina@gmail.com	S74706	Qas	$2a$10$hNIrVuoj9E52BrXvc	Computer Science

Figure 8.3: User table with password hashing implemented

Availability: Recognizing the importance of reliability, I integrated UptimeRobot for real-time system monitoring. This allowed for immediate incident response, ultimately proving and sustaining a 99% application uptime.

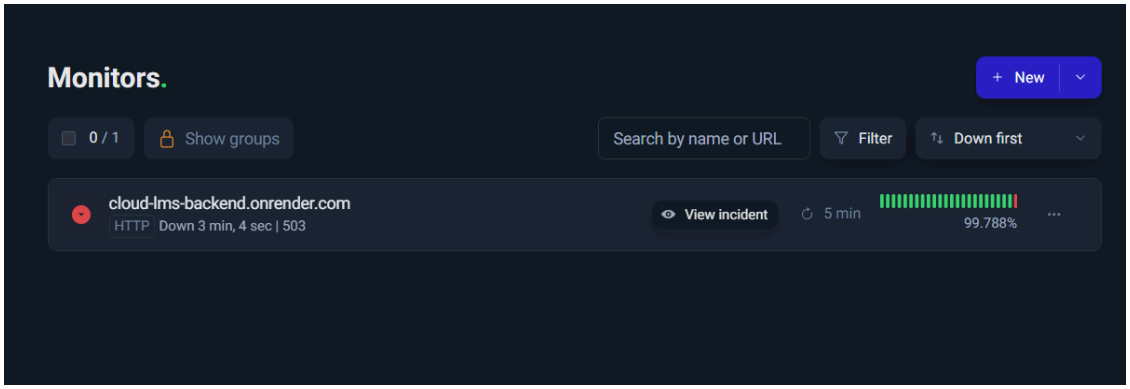

Figure 8.4: Uptime status monitoring using UptimeRobot

Maintainability: Focused on long-term sustainability by writing self-documenting code and enforcing strict architectural boundaries. This drastically reduces technical debt and makes it much easier for new developers to onboard and maintain the system.

Automated System Integration Testing was conducted after the deployment using Apache Netbean IDE, Java, Selenium and JUnit. The test simulated the flow of adding courses through login and creating course forms. The testing is executed to verify that both the frontend and backend is successfully integrated to process and store data. The result of the testing output is shown in Figure 8.5.

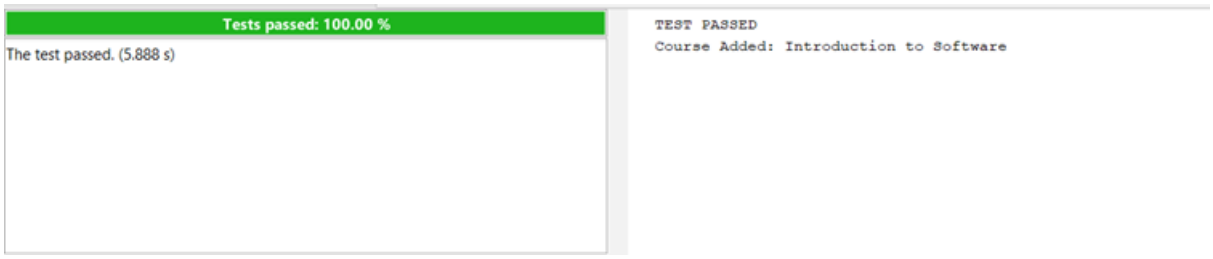

Figure 8.5: Test Execution Results for Adding a Course

9. Team Contribution

Name	Role	Contribution
Eashenraj A/L Nagarajan	Backend & Deployment Lead	Spring Boot backend development, Render deployment and configuration, Aiven MySQL cloud database setup and SSL connection configuration, application.properties environment setup, backend debugging, and API endpoint testing using Postman.
Mohammad Adam Basheer B. Mohd Azrul	Frontend & Database Lead	React frontend development using Vite, frontend component design and styling, Axios API integration connecting frontend to backend, database schema design, entity model classes, and Netlify deployment.
Ammar Haziq B. Saiful Bahri	Full Stack & Architecture Lead	Spring Boot controller and service layer implementation, Data Access Layer DAO structure using Spring Data JPA and Hibernate, frontend dashboard development for both Student and Instructor views, system architecture design, and architectural documentation and reporting.

10. Conclusion and Reflection

This project gave us a clearer picture of what building a real system actually involves compared to just learning concepts in class. Layered architecture and separation of concerns made a lot more sense once we had to apply them ourselves and decide where each piece of logic belongs and why one layer should not reach into another. The most valuable part was going through the full cycle of building, breaking, and fixing a live-deployed system. Issues like CORS errors, database connection failures, and mismatched field names between the frontend and backend were frustrating at the time, but taught us more than we expected about how the different parts of a system depend on each other.

Deploying across three separate platforms like Netlify, Render, and Aiven also made us appreciate why cloud-native design matters. Each layer could be updated independently without bringing down the whole system, which is something that is easy to read about but only really clicks when you experience it. Overall, we are satisfied with what we managed to build and deploy as a working prototype. There is still room to grow the system further, but the foundation is solid, and the architecture held up throughout the development process.

11. Reference

1. Cherif, Y. (2024, November 28). *Understanding the Layered Architecture Pattern: A Comprehensive guide*. DEV Community. https://dev.to/yasmine_ddec94f4d4/understanding-the-layered-architecture-pattern-a-comprehensive-guide-1e2j

2. *Home | Moodle.org*. (n.d.). https://moodle.org/

3. GeeksforGeeks. (2025, July 23). *Layered architecture of cloud*. GeeksforGeeks. https://www.geeksforgeeks.org/devops/layered-architecture-of-cloud/

4. Savolainen, J., & Myllarniemi, V. (2009, September). Layered architecture revisited—Comparison of research and practice. In *2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture* (pp. 317-320). IEEE.

5. Aldheleai, H. F., Bokhari, M. U., & Alammari, A. (2017). Overview of cloud-based learning management system. *International Journal of Computer Applications*, *162*(11), 41-46.

6. GeeksforGeeks. (2026, March 2). *REST API Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/node-js/rest-api-introduction/

7. Tu, Z. (2023). Research on the application of layered architecture in computer software development. *Journal of Computing and Electronic Information Management*, *11*(3), 34-38.

8. Marmsoler, D., Malkis, A., & Eckhardt, J. (2015). A model of layered architectures. *arXiv preprint arXiv:1503.04916*.

9. Jadeja, Y., & Modi, K. (2012, March). Cloud computing-concepts, architecture and challenges. In *2012 international conference on computing, electronics and electrical technologies (ICCEET)* (pp. 877-880). IEEE.

10. Muppala, M. (2025). *SQL Database Mastery: Relational Architectures, Optimization Techniques, and Cloud-Based Applications*. Deep Science Publishing.

11. Aldheleai, H. F., Bokhari, M. U., & Alammari, A. (2017). Overview of cloud-based learning management system. *International Journal of Computer Applications*, *162*(11), 41-46.

12. Jeong, J. S., Kim, M., & Yoo, K. H. (2013). A content oriented smart education system based on cloud computing. *International Journal of Multimedia and Ubiquitous Engineering*, *8*(6), 313-328.

13. *Netlify. (2019). Netlify: All-in-one platform for automating modern web projects. Netlify; Netlify. https://www.netlify.com/*

14. *Aiven - Your Trusted Data & AI Platform*. (2016). Aiven. https://aiven.io/

15. *Cloud Application Hosting for Developers | Render*. (n.d.). Cloud Application Hosting for Developers | Render. https://render.com/